Evidence-Bounded Semantic Conformance Across Heterogeneous AI-Agent Executors

TRUSCOR Private Limited - gautam@truscor.org

Gautam R. Patil

2026-08-03

Abstract

Agent executors expose heterogeneous actions, capabilities, status vocabularies, and evidence. We propose an evidence-bounded conformance contract that separates portable action intent from provider mechanics and classifies an attempted action as conformant, nonconformant, unsupported, or inconclusive. A conformant result requires an exact capability match, acceptable normalized outcome, complete side-effect accounting, required evidence roles, and SOVA-owned post-action observation; provider success alone is insufficient. The architecture retains retry lineage, cancellation, timeout, partial results, and observation provenance while keeping policy, judging, signing, and threat modeling outside adapters. On a frozen 60-case constructed corpus, the executable decision matched all labels; provider-status, exact-output, capability-only, and no-independent-observation baselines achieved 0.400, 0.417, 0.200, and 0.600 accuracy. A foreign subprocess harness exercised 30 cases without importing SOVA and did not accept false-success cases as conformant. A no-Atlas vertical study completed 24 capsules and traces across scripted and restricted local executors, including capture, playback, controlled reproduction, comparison, and offline verification. These studies show executable specification consistency on one host. A later 72-case browser, computer, and terminal broker study matched every expected outcome, accepted zero unverified provider successes, made zero non-idempotent fallback violations, and stored zero secret checkpoint inputs. Pinned live receipts exercised Playwright MCP, Windows-MCP, and a policy-blocked MELRA transport. These results do not establish cross-provider validity or secure sandboxing. The paper therefore presents a falsifiable architecture and a blind confirmatory protocol for independently authored adapters, realistic browser/computer actions, and multiple providers.

PRIVATE PREPRINT CANDIDATE - NOT CLEARED FOR PUBLIC DISTRIBUTION
Version 0.96. This document is a review artifact, not a peer-reviewed result.

Affiliation: TRUSCOR Private Limited
Contact: gautam@truscor.org
Candidate arXiv classification: cs.SE (cross-list candidates: cs.AI, cs.CR)

1. Introduction

Portable AI-behavior experiments have two layers that should not be confused. The first describes what an action means: read an artifact, invoke a tool, navigate a browser, or interact with a computer under declared authorization and side-effect constraints. The second implements that intent with a concrete provider, protocol, process, browser driver, or remote runtime. A portable artifact becomes misleading if provider completion is treated as semantic success even when the requested effect did not occur, an undeclared effect occurred, required evidence is absent, or the provider silently substituted a weaker capability.

Protocol standards address important parts of this problem. MCP negotiates protocol versions and optional capabilities. WebDriver BiDi defines asynchronous commands, results, errors, events, and command identifiers. OpenTelemetry supplies cross-system correlation and semantic conventions. Record/replay systems can isolate and replay external interactions. Runtime verification work can check an execution against a contract. None of those broad mechanisms is claimed as new here.

The narrower question is whether a portable AI-behavior system should accept a provider's success only when separately identified observations satisfy the behavioral contract, and whether a four-state decision

preserves useful uncertainty across heterogeneous executors. The proposed mechanism is an *evidence-bounded semantic conformance contract*. It is designed for the SOVA artifact model but is stated independently of any one executor.

This working paper contributes:

1. a provider-neutral contract separating intent, capability, provider outcome, evidence roles, side effects, and external observations;
2. a four-state decision that does not collapse missing evidence into either success or failure;
3. explicit retry, partial-result, cancellation, and side-effect lineage rules;
4. an experimental implementation connected to `.sova` capsules and `.sova-trace` evidence;
5. three reproducible internal studies and a negative generalization result;
6. a confirmatory protocol that must pass before a publication claim is made.

2. Scope and claim boundary

This work evaluates *observable conformance*. It does not claim to determine a model's hidden reasoning, prove an observer truthful, make ordinary host process restriction a security sandbox, or turn executor output into non-repudiable evidence. A SOVA observer is outside the adapter boundary in the software architecture; that separation is not proof of organizational or physical independence.

The candidate claim is not that adapters, capability negotiation, retries, status normalization, record/replay, trace correlation, or runtime contracts are novel. The testable delta is:

> A portable executor decision that combines exact capability alignment, evidence-role requirements, declared side-effect accounting, and separately attributed post-action observations can avoid false acceptance of provider-reported success while retaining an explicit inconclusive state when required observations are absent.

The claim is supported only if it survives independently authored adapters, multiple providers, realistic failure injection, and blinded labels.

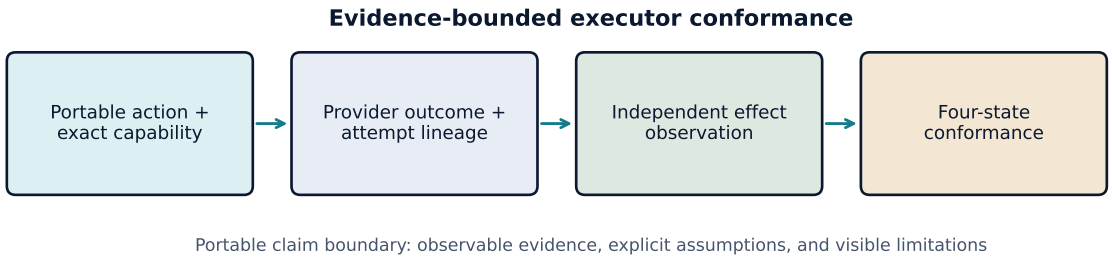


Figure 1: Conceptual architecture for Evidence-Bounded Executor Conformance. Boxes show the portable decision boundary; executor or producer mechanics remain outside the claim.

3. System model

3.1 Entities and trust boundaries

The model contains five logical entities:

- **Experiment author:** declares portable intent and expected observable effects.
- **SOVA orchestrator:** enforces authorization, selects an adapter, records the trace, and applies the conformance decision.
- **Executor adapter:** translates portable intent into provider mechanics.
- **Execution provider:** performs terminal, browser, computer, or other operations and reports an outcome.
- **Observer:** measures declared effects outside the adapter's report path.

The adapter and provider may be buggy, incomplete, overconfident, compromised, or merely semantically different from the authoring environment. The orchestrator and observer may also be wrong. The decision therefore reports a bounded conclusion under declared trust assumptions rather than objective truth.

3.2 Portable action and capability

A portable request is

`A = (id, action, inputs, timeout, retryAttempt)`.

An exact capability descriptor is

`C = (name, version, maximumSideEffect, idempotent, evidenceRoles)`.

Compatibility uses the capability identifier `name/version`. A higher version does not imply substitutability. Missing, weaker, emulated, differently scoped, and newer capabilities must each be handled explicitly. Emulation is allowed only when the experiment contract permits it and the trace records it.

3.3 Provider outcome

For each attempt, the provider returns a normalized outcome

`P = (requestId, status, sideEffect, output, evidence, verification, retryable, errorCode, limitations, failureCause)`.

Terminal status is one of `succeeded`, `failed`, `timeout`, `cancelled`, `denied`, `unsupported`, or `partial`. Failure cause is separately normalized as target, executor, policy, environment, evidence, timeout, cancellation, unsupported, unknown, or none. This prevents a transport exception, target failure, and policy denial from being silently treated as the same event.

3.4 Evidence and observations

An evidence reference identifies a role, media type, content digest, and byte size. Evidence roles describe what a referenced artifact purports to support; the role is not proof of truth.

An observation requirement is

`Q_i = (id, kind, path, expected)`.

An observed record includes a producer identity and a typed payload. Only records attributed to the declared SOVA observer set are considered when testing `Q_i`. The producer field is provenance data. Cryptographic integrity can bind it to recorded bytes, but cannot prove that the producer was honest or independent.

4. Evidence-bounded conformance decision

Let `R` be the required evidence-role set and `O` the post-action observations. The reference decision applies these rules in order:

1. If request intent does not match the contract, return `nonconformant`.
2. If no terminal provider outcome exists in the available trace prefix, return `inconclusive`.
3. If the exact required capability is absent or the provider explicitly reports unsupported semantics, return `unsupported`.
4. If terminal status, side-effect class, or required evidence roles contradict the contract, return `nonconformant`.
5. For each observation requirement, select only records attributed to an allowed observer producer.
6. If a present observation contradicts the requirement, return `nonconformant`.
7. If one or more required observations are absent and no contradiction is present, return `inconclusive`.
8. Otherwise return `conformant`.

The ordering is intentional. `unsupported` is not experimental failure; `inconclusive` is not success; and a contradiction is not erased by a later provider success.

4.1 Retry and side-effect lineage

Every retry is a new attempt linked to the earlier attempt. A successful retry does not erase a partial write, duplicate external call, orphan process, or other observed effect from a prior attempt. Automatic retry is safe only when the capability is declared idempotent or the contract supplies a specific deduplication mechanism. Cancellation and timeout describe the orchestrator's control state, not a guarantee that provider-side effects ceased.

4.2 Partial traces

The decision is prefix-aware. A trace that ends before a terminal provider outcome or required observation is available is inconclusive. It must not be reclassified as nonconformant merely because capture crashed, nor as conformant because an intermediate event resembled success.

4.3 Separation from policy and judging

Authorization, containment, safety policy, semantic judging, cryptographic verification, and oracle construction remain outside executor adapters. An adapter cannot expand authorization, sign its own conclusion as the SOVA trust root, or choose the test oracle that evaluates it.

5. Implementation

The public experimental implementation defines typed capabilities, action requests, normalized outcomes, evidence references, execution contexts, cooperative cancellation, and exact capability negotiation. It includes:

- `ScriptedExecutor`, a deterministic adapter for fixtures and fault injection;
- `RestrictedLocalExecutor`, a host-process executor limited to an explicit workspace and policy surface;
- normalized runner events and evidence references;
- budgets, timeouts, cancellation, retries, and supervised child processes;
- post-action oracles owned by the SOVA orchestration layer;
- capsule packaging, trace playback, controlled reproduction, comparison, and offline verification.

`RestrictedLocalExecutor` is deliberately not called a security sandbox. Host process controls vary by operating system and do not establish strong isolation against malicious native code.

The private research prototype embeds an experimental `x-truscor-research/0.1.0` contract in a real capsule, evaluates it from a verified trace, and emits the four-state result. Novelty-bearing research code remains private pending founder and counsel review.

6. Research questions

- **RQ1:** Does evidence-bounded conformance reduce false acceptance and false rejection compared with provider status, exact output, capability-only, and a decision without independently attributed observations?
- **RQ2:** Which components account for the result: exact capability version, evidence roles, side-effect accounting, independent observation, or retry lineage?
- **RQ3:** Does the contract preserve its decisions across executor mechanics, action families, providers, operating environments, and supported runtimes?
- **RQ4:** What end-to-end overhead does evidence capture and decision-making add relative to executor and provider latency?
- **RQ5:** Under what failure modes does the mechanism correctly remain inconclusive rather than produce false conformance or false nonconformance?

7. Internal experimental protocol

7.1 Constructed 60-case corpus

The preregistered prototype corpus contains 60 deterministic cases: 12 conformant, 24 nonconformant, 12 inconclusive, and 12 unsupported. Cases vary request alignment, provider terminal state, side-effect class, required evidence roles, observation presence, and observation contradiction.

The candidate is compared with:

1. provider status only;
2. exact output equality;
3. capability presence only; and
4. the candidate contract without the independent-observation requirement.

Primary metrics are accuracy, macro-F1, per-class precision and recall, false acceptance among nonconformant/inconclusive/unsupported truth, and false rejection among conformant truth. Because the prototype emits categorical states rather than confidence, calibration is not applicable.

7.2 Foreign subprocess harness

A line-delimited JSON provider runs in a separate process, imports no SOVA module, and returns only provider status and evidence assertions. A parent-side observer independently hashes the resulting fixture. Thirty cases contain six variants each for honest success, false success with no effect, contradictory effect, unsupported action, and missing evidence.

This is process and implementation separation, not human or organizational independence: the SOVA team authored the harness and labels.

7.3 No-Atlas vertical study

Twelve safe artifact-read cases run through `ScriptedExecutor` and `RestrictedLocalExecutor`. Each run creates a trace and capsule, then exercises inspection, playback, controlled reproduction, semantic comparison, native verification, and dependency-free verifier paths. The 24 resulting capsules and 24 traces contain no external model or Atlas dependency.

7.4 Supported-runtime repetition

The vertical study was repeated with CPython 3.11.15, 3.12.13, 3.13.12, and 3.14.3 on one Windows host. This tests supported interpreter drift but not a second machine, operating system, or independently operated environment.

7.5 Performance scope

The microbenchmark executes only the in-process four-state decision for 10,000 calls. It excludes provider time, observation, trace serialization, storage, verification, and network latency. It is diagnostic evidence, not an end-to-end performance claim.

7.6 Topic 13 broker and live-adapter study

A later frozen protocol evaluated 72 deterministic cases: browser, computer, and terminal families crossed with verified success, unverified success then fallback, retryable failure then fallback, and non-idempotent stop strata. Six cases populated every family-stratum cell. The protocol required exact outcome and attempt counts, zero unverified provider successes accepted, zero fallback after non-idempotent failure, and zero secret material persisted in checkpoints.

The engineering audit also pinned and exercised three real adapter paths. Playwright MCP 0.0.78 (Microsoft 2026) supplied a headless isolated browser session and post-action accessibility evidence. Windows-MCP 0.8.2 (CursorTouch 2026) supplied a read-only desktop snapshot with telemetry disabled. MELRA package 0.3.0-alpha.0 (S. and XAGI Lab 2026) reached its MCP transport but returned a policy-blocked action result. The latter is a retained negative result: transport availability did not become action success.

These live receipts are not part of the 72 constructed labels and are not presented as an independent adapter study. Microsoft Playwright MCP explicitly does not describe itself as a security boundary, and ordinary Windows process or profile isolation is not called a sandbox here.

8. Results

8.1 Four-state corpus

Method	Accuracy	Macro-F1
Evidence-bounded candidate	1.000	1.000
No independent observation	0.600	0.542
Exact output	0.417	0.183
Provider status	0.400	0.350
Capability only	0.200	0.083

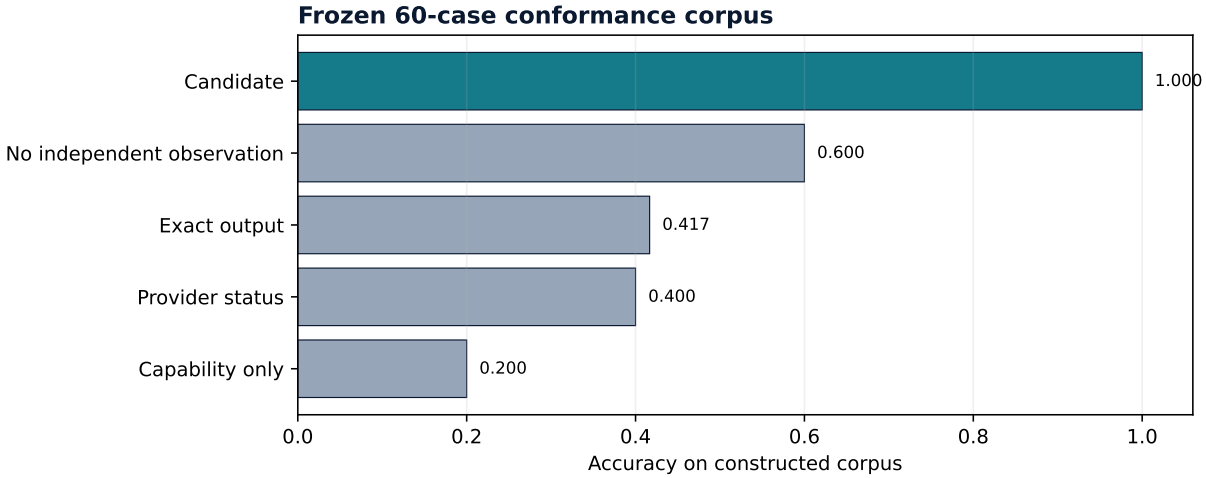


Figure 2: Constructed-corpus baseline comparison. Values are retained experiment results, not field-performance estimates.

Against the same-team constructed labels, the candidate produced zero false acceptances and zero false rejections. This perfect score is a fixture conformance result, not an estimate of real-world generalization. The cases were derived from the decision rules, so circularity is a material threat.

The observation ablation increased accuracy error from zero to 0.400. It accepted cases whose provider/evidence surface looked complete even though an independent observation was missing or contradictory. This supports the mechanism's internal logic but does not isolate every component. Capability versioning, retry lineage, and side-effect accounting still require separate held-out ablations.

8.2 Foreign subprocess harness

All 30 cases matched the declared four-state truth. None of the six false-success cases was accepted as conformant; each remained inconclusive because the required parent-side observation was absent. This removes direct dependence on the two public Executor classes, but remains a one-host, same-author, local-filesystem study.

8.3 Integrated capsule and trace study

The no-Atlas vertical study passed all 12 cases and all 24 executor runs. Native and independent verification paths accepted all expected traces and capsules; oracles, playback, packaging, controlled reproduction, and declared semantic comparisons passed. Total reference-host duration was 13.91 seconds.

In a separate seven-case integrated study, the candidate matched all constructed labels. Normalized request/outcome interaction equality matched 3/7, provider terminal completion 2/7, and exact transport bytes 1/7. A provider success contradicted by the declared SOVA observer was nonconformant; a partial prefix without a terminal provider outcome was inconclusive.

8.4 Runtime repetition and decision overhead

The stable outcome slice was byte-identical across the four tested CPython versions, with canonical digest `sha256:f27cd1c4fdcff9903bf89e41de9436dc9e930b34ffc0347b16c8fd2f494d6f6f`. All runs occurred on one Windows host.

The latest repeated decision-function benchmark measured 5.89 microseconds per call on average and 6.35 microseconds at the 95th batch percentile over 10,000 calls. Earlier measurement variation is retained in the raw artifacts. The number excludes every dominant end-to-end cost and must not be advertised as executor overhead.

8.5 Windows cancellation and cleanup falsification

A later full-suite run exposed a real Windows race: the host's `taskkill /T /F` returned access denied, after which the executor waited ten seconds before its owned-process fallback and could attempt temporary

stderr deletion while the handle was still releasing. The implementation now treats a nonzero or timed- out tree-kill command as an immediate signal to kill and wait for the owned root process, and performs bounded cleanup retries for both foreground and background temporary I/O. Two dedicated retry/failure tests, the complete 42-test executor module, and four concurrent cancellation/foreground stress lanes passed after the correction; observed background cancellation fell from about ten seconds to below one second on the reference host.

This is a lifecycle hardening result, not proof of arbitrary Windows process- tree containment. Without a job-object or stronger isolation backend, a child that successfully creates independently surviving descendants may exceed the restricted host executor's guarantees. The existing "not a security sandbox" boundary therefore remains mandatory.

8.6 Topic 13 broker and live-adapter result

All 72 constructed cases matched their expected terminal state and attempt count. The broker recorded 36 provider-only successes but accepted zero unverified provider successes as completed; 54 successes were backed by the fixture's declared post-action verification. There were zero non-idempotent fallback violations and zero checkpoint secret-persistence violations.

The browser receipt produced a provider result plus separately identified post-action evidence. The Windows receipt demonstrated a read-only observation path. MELRA produced a policy-blocked outcome despite successful protocol contact. Together they falsify a simple "connected means succeeded" assumption while leaving the stronger hypothesis unresolved: the observers and expected labels are still team controlled, and the live receipts do not constitute a blinded cross-provider conformance corpus.

9. Analysis

The internal result suggests three useful design properties.

First, provider completion and semantic conformance are distinct. A provider can truthfully report that its command completed while the requested observable effect is missing or incompatible. Second, missing evidence should not be forced into a binary success/failure decision. The inconclusive state preserves the difference between a contradicted effect and an unobserved effect. Third, portable comparison requires more than byte equality: two providers can produce different mechanics and transport bytes while satisfying the same declared observable contract.

The studies do not show that the candidate is superior in real systems. The 60-case labels reflect the mechanism's own rules, the foreign harness was team authored, and the vertical slice exercises only safe local artifact reads. The result is therefore best interpreted as an executable specification and falsifiable hypothesis.

10. Prior art and differentiating boundary

10.1 Protocol capability and command semantics

MCP revision 2025-11-25 (Model Context Protocol Authors 2025) standardizes JSON-RPC-based protocol negotiation and capability surfaces for agent tools and resources. WebDriver BiDi's (World Wide Web Consortium 2026) 1 June 2026 Working Draft specifies asynchronous browser commands, results, errors, events, session capabilities, and out-of-order completion. These standards are direct baselines for adapter capability and lifecycle behavior. SOVA should map to them, not replace them.

10.2 Telemetry and provenance

OpenTelemetry semantic conventions 1.43.0 (OpenTelemetry Authors 2026) define common operation attributes, spans, events, resources, and protocol-specific conventions. They support correlation and analysis but do not make a provider's behavioral claim true. SOVA exports to and imports from this telemetry model with explicit loss accounting.

10.3 Replay and execution verification

`agrepl` (Mudasiru 2026) intercepts external interactions, isolates replay, defines request-key matching, and reports deterministic replay results over five workloads. Exact transport playback is therefore not a novelty

claim for SOVA. Shepherd (Yu et al. 2026) and execution-lineage work (Rosen and Rosen 2026) likewise make typed trace, state lineage, branching, and replay active prior-art areas. Proof of Execution (Rhodes and Kang 2026) formalizes contract, causal event, validator, authority, and replay guarantees for governed agent actions. These systems constrain any broad claim over contract-plus-trace verification.

The remaining research hypothesis is narrower: four-state semantic conformance conditioned on exact capability, evidence roles, side-effect accounting, and observation provenance across materially different providers. No novelty conclusion follows until direct experiments and a qualified prior art review are complete.

11. Threat model and failure modes

11.1 Malicious or faulty adapter

An adapter may fabricate status, evidence references, limitations, or capabilities. An external observation can reveal some false-success cases, but cannot detect effects outside the observer's coverage. A malicious adapter may also attack the observer or recorder. Process separation is not a security boundary.

11.2 Malicious or faulty observer

The observer can be incomplete, compromised, causally confused, or controlled by the same party as the adapter. Producer identity and signatures establish provenance only within the declared key and recorder assumptions. They do not prove independence or truth.

11.3 Duplicate and delayed effects

Timeout or cancellation can race with provider-side completion. Retry can duplicate non-idempotent effects. The trace must retain all attempts and observed effects; a later success cannot erase earlier effects.

11.4 Capability equivocation

Providers may advertise a capability but perform a weaker operation, or change semantics without changing the version. Exact identifiers reduce silent substitution but do not solve dishonest advertising. Conformance therefore still depends on observation coverage.

11.5 Evidence tampering

Content digests and signed manifests can make mutation detectable within a tested trust policy. They do not prove the original execution happened as claimed. This paper makes no unforgeability or non-repudiation claim.

12. Threats to validity

Construct validity

The contract treats declared observable requirements as ground truth. Real AI behaviors may be underspecified, distributional, or ethically unsafe to replay. Observer paths may not capture the intended semantic property.

Internal validity

The team authored the mechanism, fixtures, and labels. Perfect constructed corpus scores may reflect rule-derived cases. The subprocess boundary does not remove author bias.

External validity

No independent adapter, browser executor, computer-use executor, Atlas adapter, remote sandbox, second provider, or second independently operated machine has passed the study. Terminal-like artifact reads do not establish portability to interactive systems.

Statistical validity

The internal sample is deterministic and balanced by construction. It cannot estimate field prevalence or provider failure rates. The confirmatory study must report per-stratum intervals and retain unsupported/inconclusive outcomes.

Security validity

The current process boundary and host restrictions do not withstand an arbitrary malicious native program. Observers are not assumed incorruptible.

13. Confirmatory protocol before submission

The submission gate requires all of the following:

1. An adapter authored by a reviewer who did not receive private SOVA implementation code. The reviewer receives only the public contract, conformance vectors, and a handoff form.
2. Terminal, browser, and computer action families across at least two materially different execution providers.
3. Cases for honest success, false success, false failure, unsupported action, weaker capability, evidence omission, contradictory observation, retry, duplicate side effect, timeout, cancellation, crash, partial result, and orphan process.
4. SOVA-owned observers kept outside adapters and independently reviewed for coverage and causal ambiguity.
5. Blinded expected labels. Candidate predictions are digest-frozen before the sealed labels are released.
6. Direct comparisons against provider status, exact output, capability-only, the no-observation ablation, one mature executor protocol, and one current record/replay implementation.
7. At least 30 cases per action family and adequate representation of every failure stratum. Report exact counts rather than hiding small cells in an aggregate.
8. Accuracy, macro-F1, per-class precision/recall, false conformance, false nonconformance, inconclusive rate, unsupported rate, effect-accounting errors, recovery, and per-stratum confidence intervals.
9. End-to-end overhead including executor, observation, trace, storage, verification, and replay cost. The microbenchmark is not substituted.
10. Ablations for observation provenance, evidence roles, side-effect class, exact capability version, retry lineage, and partial-trace handling.

Predeclared kill criteria:

- any undetected false conformance in a safety-critical or destructive stratum;
- macro-F1 below the strongest baseline without a compensating, predeclared safety benefit;
- inability to distinguish missing evidence from contradiction;
- materially different decisions across providers caused solely by adapter mechanics that the portable contract declares irrelevant;
- observer design that depends on private provider status rather than an independently measurable effect.

Failure is retained as a publishable negative result if the protocol was not changed after label access.

14. Reproducibility and artifact plan

The internal studies retain private source, preregistration, raw JSON results, environment fingerprints, exact dependency versions, and test commands. The external reviewer packet will contain:

- the public executor contract and safe conformance vectors;
- adapter-author instructions and a declaration of information received;
- separate case and sealed-label files;
- a prediction-freeze digest receipt;
- machine-readable provider, environment, capability, action, and failure metadata;
- a scoring script that refuses case-set or digest mismatch;
- a limitations and conflict-of-interest form.

Real credentials, client data, confidential evidence, and private Atlas/MELRA material must not enter the packet. Dheeraj S built the Atlas MCP project now named MELRA and is acknowledged for that external

executor work. He is not a SOVA paper author or candidate inventor on the present record because no documented contribution to this SOVA mechanism has been established.

15. Conclusion

The contract makes a provider's status one input to conformance rather than its proof. The retained studies show that capability, output, and provider status are insufficient on constructed cases, while also showing that same-team fixtures and ordinary process restriction cannot establish real cross-provider security. Independent adapters, realistic interaction families, blinded labels, and end-to-end measurements must now support or falsify the hypothesis.

Declarations

Ethics and dual-use statement

The reported machine experiments use synthetic or public research fixtures and safe local actions. They do not target a live third party, use client data, store real credentials, or claim access to private model thoughts or hidden chain-of-thought. The work can support security testing and therefore has dual-use potential. Unsafe reproduction remains subject to explicit authorization, containment, and human review; operational exploit details and novelty-sensitive implementations are not part of this private review package.

Data and artifact availability

The generic SOVA implementation, public specifications, safe examples, and tests are available in the SOVA-OSS repository. Private experiment results, preregistrations, source digests, build provenance, and review packets are retained by TRUSCOR Private Limited. Third-party source artifacts whose redistribution rights are unclear are not redistributed. A release artifact must be assembled only after rights, privacy, security, and IP review.

Author contributions and AI-assisted work

Gautam R. Patil conceived the SOVA project, specified the research direction and safety constraints, and directed this work. Software implementation, experiment execution, analysis support, literature discovery, and drafting used OpenAI Codex under his direction. Gautam R. Patil is the sole proposed human author and must verify the final text, citations, figures, data, and venue disclosure before submission. No AI system is listed as an author or inventor.

Funding and competing interests

This work was conducted for TRUSCOR Private Limited. No external funding was used for the reported machine experiments. The author's affiliation with TRUSCOR Private Limited is a relevant non-financial and potential commercial interest. The exact legal entity name and any additional interests must be confirmed before submission.

References

CursorTouch. 2026. *Windows-MCP*. https://github.com/CursorTouch/Windows-MCP.

Microsoft. 2026. *Playwright MCP*. https://github.com/microsoft/playwright-mcp.

Model Context Protocol Authors. 2025. *Model Context Protocol Specification, Revision 2025-11-25*. https://modelcontextprotocol.io/specification/2025-11-25.

Mudasiru, Rasheed. 2026. *Deterministic Replay for AI Agent Systems*. https://arxiv.org/abs/2607.16200.

OpenTelemetry Authors. 2026. *OpenTelemetry Semantic Conventions 1.43.0*. https://opentelemetry.io/docs/specs/semconv/.

Rhodes, James, and George Kang. 2026. *Proof of Execution: Runtime Verification for Governed AI Agent Actions*. https://arxiv.org/abs/2607.05397.

Rosen, Josh, and Seth Rosen. 2026. *From Agent Loops to Deterministic Graphs: Execution Lineage for Reproducible AI-Native Work*. https://arxiv.org/abs/2605.06365.

S., Dheeraj, and XAGI Lab. 2026. *MELRA*. https://github.com/XAGI-Lab/melra.

World Wide Web Consortium. 2026. *WebDriver BiDi*. https://www.w3.org/TR/2026/WD-webdriver-bidi-20260601/.

Yu, Simon, Derek Chong, Ananjan Nandi, et al. 2026. *Enabling Programmable Meta-Agents via Reversible Agentic Execution Traces*. https://arxiv.org/abs/2605.10913.